# Manufactured solutions for the bidomain equations

**Construction of the exact potentials, sources and gauge**

Simão Castro

## Table of contents

**1 The problem** **2**
- 1.1 Domain and governing equations . . . 2
- 1.2 Manufactured ionic closure . . . 2
- 1.3 Gauge of the extracellular potential . . . 3
- 1.4 Nomenclature . . . 3
- 1.5 Reading and regenerating . . . 5
- 1.6 Creating a new manufactured solution . . . 6

**2 FDA cube** **8**
- 2.1 Description and limitations . . . 8
- 2.2 Problem instance . . . 9
- 2.3 Construction . . . 9
- 2.4 Exact solution . . . 10
- 2.5 Self-checks . . . 11
- 2.6 Figures . . . 12
- 2.7 Regeneration . . . 13

**3 Sealed spherical shell** **13**
- 3.1 Description and limitations . . . 13
- 3.2 Problem instance . . . 13
- 3.3 Construction . . . 14
- 3.4 Exact solution . . . 16
- 3.5 Self-checks . . . 16
- 3.6 Figures . . . 17
- 3.7 Regeneration . . . 19

**References** **20**

# 1 The problem

This document describes how each manufactured solution of the bidomain equations used by cardiacFoam is built. In the method of manufactured solutions (Roache 2002) the exact potentials are chosen first and the sources that make them satisfy the equations are derived from them. The document gives the exact potentials, the sources, the gauge of the extracellular potential and the checks that each construction passes. It states the problems only; it contains no solver output. Every displayed field, tensor, source and coefficient below is printed by the sympy generator of its problem, `fda_bidomain_mms.py` or `shell_bidomain_mms.py`, which also writes the NumPy reference module and, for the shell, the C++ reference header of cardiacFoam. A reader with this document and the generator folder can rebuild each problem in another code.

## 1.1 Domain and governing equations

In the myocardium $\Omega \subset \mathbb{R}^d$, $d \in \{1, 2, 3\}$, with sealed boundary $\partial\Omega$ and outward unit normal $\mathbf{n}$, the bidomain equations for the intracellular and extracellular potentials $\phi_i$, $\phi_e$ and the transmembrane potential $V_m = \phi_i - \phi_e$ are

$$\begin{cases} \chi C_m \dfrac{\partial V_m}{\partial t} = \nabla \cdot (\mathbf{G}_i \nabla \phi_i) - \chi C_m I_{\text{ion}} + I_{\text{app}} & \text{in } \Omega, \\ 0 = \nabla \cdot (\mathbf{G}_i \nabla \phi_i) + \nabla \cdot (\mathbf{G}_e \nabla \phi_e) - s_e & \text{in } \Omega, \\ \mathbf{n} \cdot \mathbf{G}_i \nabla \phi_i = 0, \quad \mathbf{n} \cdot \mathbf{G}_e \nabla \phi_e = 0 & \text{on } \partial\Omega, \end{cases}$$

with the ionic closure $I_{\text{ion}} = I(V_m, \mathbf{w})$, $\mathrm{d}\mathbf{w}/\mathrm{d}t = \mathbf{F}(V_m, \mathbf{w})$, and initial data $V_m(\mathbf{x}, 0)$, $\mathbf{w}(\mathbf{x}, 0)$. cardiacFoam (`bidomainSolver`) solves the system in the unknowns $V_m$ and $\phi_e$:

$$\begin{aligned} \chi C_m \frac{\partial V_m}{\partial t} &= \nabla \cdot (\mathbf{G}_i \nabla V_m) + \nabla \cdot (\mathbf{G}_i \nabla \phi_e) - \chi C_m I_{\text{ion}} + I_{\text{app}}, \\ \nabla \cdot ((\mathbf{G}_i + \mathbf{G}_e) \nabla \phi_e) &= -\nabla \cdot (\mathbf{G}_i \nabla V_m) + s_e. \end{aligned}$$

The second equation with the two sealed walls is a pure Neumann problem: $\phi_e$ is fixed up to a constant, and a solution exists only if $\int_\Omega s_e \, \mathrm{d}V = 0$. Every problem below prescribes the exact fields at $t = 0$ as initial data, and both sealed-wall conditions are satisfied by the exact potentials themselves. The units are those of the monodomain document: SI units for the continuum equations, and fields written in case units on the normalised coordinates $x/(1\,\mathrm{m})$ and $t/(1\,\mathrm{s})$.

## 1.2 Manufactured ionic closure

Both problems use the manufactured ionic model of Pathmanathan and Gray (2014), with state $(V_m, u_1, u_2, u_3)$. The solver supplies $V_m$; the model advances the auxiliary states with $V_m$ held fixed, at the rates

$$\dot{u}_1 = u_2^2 \, (V_m - u_3) \left( -\frac{V_m}{2} + \frac{u_1}{2} + \frac{u_3}{2} \right) + u_2^2 \, (-V_m + u_1 + u_3)^2$$

$$\dot{u}_2 = u_2^3 \, (V_m - u_1 - u_3)$$

and $\dot{u}_3 = 0$. The model returns

$$J_{\text{ion}} = \frac{\beta\,(V_m - u_3)}{\chi} - \frac{C_m u_2^2\,(V_m - u_3)\,(-V_m + u_1 + u_3)}{2}$$

and the solver divides it by $C_m$,

$$I_{\text{ion}} = \frac{J_{\text{ion}}}{C_m}$$

The FDA cube uses the ionic model `bidomainFDAManufactured`, whose $\beta_d$ is computed from the two conductivities (Section 2); the shell uses `monodomainFDAManufactured` with the rounded $\beta = \beta_3 = -8.6$ of the monodomain document. With $u_3 = 0$ and auxiliary states that make $(u_1 - V_m)u_2^2 = 1/(1+t)$, the ionic term of the $V_m$ equation is

$$-\chi C_m I_{\text{ion}} = \chi C_m \frac{V_m}{2(1+t)} - \beta V_m,$$

which cancels the time derivative of $V_m = \sqrt{1+t}\,F$ and leaves $-\beta V_m$.

## 1.3 Gauge of the extracellular potential

The exact potentials are defined up to a common constant $c$. Each problem fixes $c$ by the value $\phi_{e,\text{ref}}$ (`phiEReferenceValue`, 0 in both tutorials) of the exact $\phi_e$ at the point $\mathbf{x}_{\text{ref}}$ (`phiERefPoint`); the solver sets the same value in the cell that contains $\mathbf{x}_{\text{ref}}$. The verifiers report two kinds of $\phi_e$ error. The rows `phiE` and `phiI` (FDA cube) compare with the exact field of this constant; their error contains the difference between the point value and the value of the reference cell. The rows `phiE_gauge` and `phiI_gauge` compare with the exact field shifted by the volume mean of the $\phi_e$ error,

$$\delta_e = \frac{\sum_c V_c\,(\phi_{e,c} - \phi_e(\mathbf{x}_c))}{\sum_c V_c}, \qquad \phi_e^{\text{gauge}} = \phi_e + \delta_e, \quad \phi_i^{\text{gauge}} = \phi_i + \delta_e,$$

with $V_c$ and $\mathbf{x}_c$ the volume and centre of cell $c$; the gauge rows do not depend on $\mathbf{x}_{\text{ref}}$. Every error norm is volume weighted: $L_1 = \sum_c V_c|e_c| / \sum_c V_c$, $L_2 = (\sum_c V_c e_c^2 / \sum_c V_c)^{1/2}$, $L_\infty = \max_c |e_c|$.

## 1.4 Nomenclature

The code name is the key of the cardiacFoam case dictionaries or the name used in the generators and the NumPy reference modules. "Case units" marks a quantity defined only through the manufactured construction, with no physical calibration.

| Symbol | Meaning | Unit | Code name |
|---|---|---|---|
| $\mathbf{x} = (x, y, z)$ | position, normalised by 1 m | m | `x`, `y`, `z` |
| $d$, $\Omega$, $\mathbf{n}$ | spatial dimension; the domain; outward unit normal of $\partial\Omega$ | –; –; – | `dim`, `d` |
| $t$ | time, normalised by 1 s | s | `t` |
| $\phi_i$, $\phi_e$ | intracellular and extracellular potentials | V | `phiI`, `phiE` |

| Symbol | Meaning | Unit | Code name |
|---|---|---|---|
| $V_m$ | transmembrane potential, $\phi_i - \phi_e$ | V | `Vm` |
| $\mathbf{G}_i$, $\mathbf{G}_e$ | intracellular and extracellular conductivity tensors | S m$^{-1}$ | `conductivityIntracellular`, `conductivityExtracellular`; `Gi`, `Ge` (shell generator) |
| $G_{i,mm}$, $G_{e,mm}$ | diagonal entries of $\mathbf{G}_i$, $\mathbf{G}_e$ on axis $m$ (FDA cube) | S m$^{-1}$ | – |
| $H_{mm}$ | $G_{i,mm}G_{e,mm}/(G_{i,mm} + G_{e,mm})$ | S m$^{-1}$ | `Gm` (C++) |
| $\kappa$ | $G_{i,xx}/(G_{i,xx} + G_{e,xx})$ | – | `k` (C++) |
| $k_d$ | $\sum_{m \le d} G_{i,mm}(m\pi)^2$, eigenvalue of $-\nabla \cdot (\mathbf{G}_i \nabla \cdot)$ for $F_d$ | s$^{-1}$ (case units) | – |
| $\chi$, $C_m$ | membrane surface-to-volume ratio; capacitance per unit area | m$^{-1}$; F m$^{-2}$ | `chi`; `cm` |
| $I_{\text{ion}}$, $J_{\text{ion}}$ | capacitance-normalised ionic current; current returned by the ionic model | V s$^{-1}$; A m$^{-2}$ (case units) | (solver side); `Iion` |
| $I_{\text{app}}$ | applied volumetric current of the $V_m$ equation | A m$^{-3}$ | `sourceField` |
| $s_e$ | manufactured source of the $\phi_e$ equation | A m$^{-3}$ | `addManufacturedPhiESource` |
| $u_1, u_2, u_3$ | auxiliary states of the manufactured ionic model | case units | `u1`, `u2`, `u3` |
| $\beta$, $\beta_d$ | reaction coefficient of the manufactured ionic model | case units | `Beta` |
| $F_d$, $G_d$ | spatial factors of the FDA field and of its auxiliary states | case units | `F_d<d>`, `G_d<d>` |
| $c$ | gauge constant of $\phi_e$ and $\phi_i$ | V | `shift` |
| $\mathbf{x}_{\text{ref}}$, $\phi_{e,\text{ref}}$ | reference point and value of $\phi_e$ | m; V | `phiERefPoint`, `phiEReferenceValue` |
| $\delta_e$ | volume mean of the $\phi_e$ error | V | `phiE gauge offset` |
| $\rho_p$, $\rho_e$ | residual coefficients of the $V_m$ and $\phi_e$ equations (FDA cube) | s$^{-1}$ (case units) | – |
| $r_1$, $r_2$ | inner and outer radius of the shell | m | `r1`, `r2` |
| $\lambda$, $\omega$ | transmural coordinate; function vanishing on both spheres | – | `lam`, `om` |
| $\mathbf{f}, \mathbf{s}, \mathbf{n}_s$ | fibre, sheet and sheet-normal directions, common to $\mathbf{G}_i$ and $\mathbf{G}_e$ | – | (shell generator) |
| $\mathbf{e}_a, \mathbf{e}_b$ | fixed frame of the shell fibre rotation | – | (shell generator) |
| $\sigma_{i,f}, \sigma_{i,s}, \sigma_{i,n}$ | eigenvalues of $\mathbf{G}_i$ along $\mathbf{f}$, $\mathbf{s}$, $\mathbf{n}_s$ | S m$^{-1}$ | `gil`, `git`, `gin` |

| Symbol | Meaning | Unit | Code name |
|---|---|---|---|
| $\sigma_{e,f}, \sigma_{e,s}, \sigma_{e,n}$ | eigenvalues of $\mathbf{G}_e$ along $\mathbf{f}$, $\mathbf{s}$, $\mathbf{n}_s$ | S m$^{-1}$ | `gel`, `get`, `gen` |
| $\theta_0, \theta_1$; $\gamma_1, \gamma_2$ | fibre angle at the inner wall and its transmural increment; rotation angles of the fixed frame | deg | `theta0`, `theta1`; `gamma1`, `gamma2` |
| $F_i$, $F_e$ | spatial factors of $\phi_i$ and $\phi_e$ (shell) | case units | `Fi`, `Fe` |
| $\hat{h}_i$, $\hat{h}_e$; $k$ | unsealed fields of $F_i$ and $F_e$; their wave number | case units; m$^{-1}$ | `hhat`, `ehat`; `k` |
| $a_1, a_2, a_3$; $b_1, \ldots, b_4$ | coefficients of $\hat{h}_i$; of $\hat{h}_e$ | case units | `hA[0..2]`; `fA[0..3]` |
| $m_{i,j}$, $m_{e,j}$; $m_i$, $m_e$ | wall correction of sphere $j$; its blend | – | (shell generator) |
| $d_j$, $\mathbf{n}_j$ | $\partial_n \omega$ on sphere $j$; unit normal of sphere $j$ out of the heart | – | (shell generator) |
| $g$ | positive spatial factor of the shell auxiliary states | case units | `g` |

## 1.5 Reading and regenerating

The generators live in `tutorials/manufacturedSolutions/generators/` of the cardiacFoam repository. Each problem has one sympy generator `<case>_mms.py` that writes

- `reference/<case>.py`, a NumPy-only module with the exact fields and sources;
- `latex/<case>_*.tex`, the fragments printed in this document.

Set up the environment once and run a generator from that folder:

```
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
.venv/bin/python fda_bidomain_mms.py --emit
.venv/bin/python fda_bidomain_mms.py --self-check
```

`--emit` runs the self-checks first and writes nothing if one fails; `--self-check` runs them alone. `shell_bidomain_mms.py` also takes `--check PATH` and `--write PATH`, which compare or rewrite its C++ header and are needed only by cardiacFoam developers. `docs/bidomain/bidomain_reference.py` holds what depends on the tutorial cases: `--self-check` asserts that their dictionaries carry the conductivities and reference points of the generators, and `--run-check KIND=CASE ...` (`KIND` is `fda` or `shell`) compares the exact fields with a finished run (Section 2.5, Section 3.5) and writes nothing. The document is built with

```
quarto render docs/bidomain/bidomain_mms.qmd --to pdf
```

and its figures with `python docs/bidomain/figures.py`, which takes `--draft`, `--outdir DIR` and `--mesh-case NAME=PATH` (names `fda_cube_hex`, `fda_cube_tet`, `shell_hex`, `shell_tet`) as the monodomain figure script does.

## 1.6 Creating a new manufactured solution

A new manufactured solution is a new generator in `tutorials/manufacturedSolutions/generators/`. The steps below go from a copy of the template to a case that cardiacFoam runs and checks.

### 1.6.1 Copy the template

`template_mms.py` is the smallest complete generator: a passive monodomain field on the unit square with a diagonal conductivity and no ionic current,

$$V_m = \sqrt{t+1}\cos(\pi x)\cos(\pi y)$$

$$S = \left(\frac{\chi C_m}{2\sqrt{t+1}} + \pi^2(\sigma_x+\sigma_y)\sqrt{t+1}\right)\cos(\pi x)\cos(\pi y)$$

$$\chi = 3, \quad C_m = 2, \quad \sigma_x = 0.13, \quad \sigma_y = 0.0176$$

Copy it and its test, rename both, and edit every step marked `REPLACE: REFERENCE_NAME`, the coordinates, the constants, `exact_field`, `operator`, the self-checks and the emitted fragments.

```
cp template_mms.py mycase_mms.py
cp test_template_mms.py test_mycase_mms.py
```

### 1.6.2 Choose the exact field

Choose the field so that the boundary condition holds by construction; the solver's boundary treatment is then measured, not assumed. The generators of this folder use four patterns:

- a field whose gradient vanishes on every face, so every conormal flux is zero for any tensor (rotated cube, $\sin^2$ factors);
- a cosine eigenfunction of the operator with zero normal derivative on the faces of a box (FDA cube, the template);
- a sealing correction $F = \hat{h} + \omega m$ on a curved wall: $\omega$ vanishes on the walls and $m$ cancels the conormal derivative of $\hat{h}$ there (the "Sealed spherical shell" section of this document);
- Dirichlet data plus pinned cells: the exact field is imposed on every face and on a layer of cells, so no flux condition is needed (Cartesian eikonal); this leaves the wall treatment untested.

Keep every factor smooth beyond the domain: polyhedral meshes put cell and face centres slightly outside a curved wall.

### 1.6.3 Derive the source

Write the operator the solver implements, with its signs and scalings, and take the source as its residual on the exact field, symbolically. For the monodomain equation that means the full tensor divergence $\nabla\cdot(\boldsymbol{\sigma}\nabla V_m)$, including the derivatives of a spatially varying tensor, the $\chi C_m$ scaling of the time derivative, and the capacitance-normalised ionic current $I_{\text{ion}} = J_{\text{ion}}/C_m$ that enters as $-\chi C_m I_{\text{ion}}$. A source derived for a different convention converges to the wrong field.

### 1.6.4 Write the self-checks first

Each check fails before anything is written. The traps met in this folder, with the check that catches each:

| trap | check |
|---|---|
| residual nonzero because a constant is rounded | keep constants as exact rationals; where a rounded constant is part of the problem, check the residual against a stated tolerance (FDA $\beta_d$) |
| conormal flux not zero on a curved wall | evaluate $\mathbf{n} \cdot \boldsymbol{\sigma} \nabla F$ on each wall at high precision (30 digits) |
| denominator that crosses zero | minimum of every denominator over the domain, extended past the walls, is positive |
| eikonal critical points | locate them; the Morse count (minima − saddles + maxima) on each sphere is 2 and every point is non-degenerate |
| electrode or quadrature node on a singular point | a coincident point contributes zero, with the solver's `VSMALL` guard reproduced in the generator |
| float sums that change with the Python version | `math.fsum` for every sum of floats |
| cancellation in a quadrature or finite-difference check | scale the tolerance by the sum of the absolute terms, not by the result |
| printed constant that hides digits | `printed_constant(value, digits)` asserts the printed decimal round-trips |

Add a finite-difference evaluation of the operator on the exact field and compare it with the source; it catches an operator written for the wrong convention, which the symbolic residual cannot.

### 1.6.5 Emit and document

`python mycase_mms.py --emit` writes `reference/mycase.py` and `latex/mycase_*.tex`. Add a section to the document of the equation by copying an existing case section and keeping its order: Description and limitations, Problem instance, Construction, Exact solution, Self-checks, Figures, Regeneration. Print every formula by including its fragment from `../../latex/`, as the existing sections do; the release test fails if an emitted fragment is included nowhere. Add the figures as a function of `docs/<equation>/figures.py`, add the generator to `GENERATORS` of `test_release.py` and to `CLI_GENERATORS` of `test_mms_codegen.py`, and render:

```
python docs/monodomain/figures.py
quarto render docs/monodomain/monodomain_mms.qmd --to pdf
```

### 1.6.6 Use it in cardiacFoam

1. Write or extend a verifier in `src/verificationModels/<equation>Verification/` that evaluates the reference on the mesh and reports the error norms.
2. Generate its reference header with `--write PATH`: either a whole header, or the lines between `// BEGIN GENERATED <name>` and `// END GENERATED <name>` of a hand-written one. Map the header name to its region function in `HEADERS` of the generator (`HEADERS = {}` in the template).

3. Add the case to `parity/mmsParity.C` and `CASES` of `parity/test_parity.py`, and a `test_parity_<case>` to the generator test that compares the compiled header with `reference/mycase.py`.
4. Add a tutorial case in `tutorials/manufacturedSolutions/` with `Allrun`, a `regression/regressionTest.sh` on `tutorials/regression/lib.sh` and its reference, and mark it in the case table of `tutorials/README.md`.
5. Check the header, the parity and the regression:

```
python mycase_mms.py --check PATH
python -m pytest parity/test_parity.py test_mycase_mms.py
```

   then run `regression/regressionTest.sh` of the tutorial case.

### 1.6.7 Reproduce

A new generator is reproduced like the existing ones: set up the environment, run `--self-check` and `--emit`, the tests, the slow release test and the renders, as in the previous section. The release test then runs the new generator from a fresh environment and checks that its emitted files are unchanged.

# 2 FDA cube

## 2.1 Description and limitations

The bidomain problem of Pathmanathan and Gray (2014), part of the FDA regulatory science tool for cardiac electrophysiology (U.S. Food and Drug Administration, Center for Devices and Radiological Health 2023), on the unit cube $[0,1]^d$, $d = 1, 2, 3$. The transmembrane potential is the FDA monodomain field and the extracellular potential is a fixed multiple of it. The case exercises the elliptic $\phi_e$ solve, the parabolic $V_m$ solve, their coupling through $\nabla \cdot (\mathbf{G}_i \nabla \phi_e)$ and $\nabla \cdot (\mathbf{G}_i \nabla V_m)$, the gauge of $\phi_e$, the ionic coupling and the time integration, on hexahedra in one, two and three dimensions and on Delaunay tetrahedra in three.

Both tensors are diagonal and $\mathbf{G}_e$ is proportional to $\mathbf{G}_i$, so $\mathbf{G}_i + \mathbf{G}_e$ is a multiple of $\mathbf{G}_i$, $\phi_e$ is a multiple of $V_m$ at every time and the elliptic equation needs no source. The exact field has a zero normal derivative on every face and the face normals are eigenvectors of both tensors, so both conormal fluxes vanish because of the field. The case therefore does not exercise unequal anisotropy ratios, off-diagonal conductivity, a $\phi_e$ source or its compatibility, or a wall flux reconstruction; the sealed shell (Section 3) does.

The residual of the equations is not zero. The tutorial dictionary gives $\mathbf{G}_e$ to the nine decimals of $\mathbf{G}_i$, so the ratio $G_{i,mm}/(G_{i,mm} + G_{e,mm})$ equals $\kappa$ to about $10^{-9}$ on the $y$ and $z$ axes and exactly on the $x$ axis. On the exact fields the residuals of the $V_m$ and $\phi_e$ equations are $\rho_p V_m$ and $\rho_e V_m$ (both written as left minus right side of the forms above), with

$$\rho_p = \sum_{m=1}^{d} \left((1-\kappa) G_{i,mm} - H_{mm}\right) (m\pi)^2, \qquad \rho_e = \sum_{m=1}^{d} \left(\kappa \left(G_{i,mm} + G_{e,mm}\right) - G_{i,mm}\right) (m\pi)^2,$$

and relative to $k_d$, the size of $\nabla \cdot (\mathbf{G}_i \nabla V_m)/V_m$,

$$
\begin{aligned}
d = 1: \quad & \frac{|\rho_p|}{k_d} = 0, \quad \frac{|\rho_e|}{k_d} = 0, \\
d = 2: \quad & \frac{|\rho_p|}{k_d} = 1.1 \times 10^{-9}, \quad \frac{|\rho_e|}{k_d} = 1.6 \times 10^{-9}, \\
d = 3: \quad & \frac{|\rho_p|}{k_d} = 1.4 \times 10^{-9}, \quad \frac{|\rho_e|}{k_d} = 2.0 \times 10^{-9}
\end{aligned}
$$

## 2.2 Problem instance

Unit cube $[0,1]^d$, all faces sealed, the diagonal conductivities

$$
\mathbf{G}_i = \begin{bmatrix} 0.111453302 & 0 & 0 \\ 0 & 0.12158542 & 0 \\ 0 & 0 & 0.030396355 \end{bmatrix}
$$

$$
\mathbf{G}_e = \begin{bmatrix} 0.046165469 & 0 & 0 \\ 0 & 0.05036233 & 0 \\ 0 & 0 & 0.012590582 \end{bmatrix}
$$

the constants, with $\kappa$ and $\beta_d$ computed from the dictionary values,

$$
\begin{gathered}
C_m = 2, \quad \chi = 3, \quad \kappa = 0.707106782352, \\
\beta_1 = -0.3221825394, \quad \beta_2 = -1.728069986, \quad \beta_3 = -2.518881653
\end{gathered}
$$

and the gauge

$$
\mathbf{x}_{\mathrm{ref}} = (0.125, 0.05, 0.05), \quad \phi_{e,\mathrm{ref}} = 0
$$

In cardiacFoam:

- tutorial `manufacturedSolutions/bidomain`, options `-dim 1|2|3` (`-mesh hex`, the default) and `-mesh tet` (three dimensions);
- myocardium solver `bidomainSolver` (`sealedHeartBoundary false`, `conductivitySource uniform`);
- ionic model `bidomainFDAManufactured`, which rejects tensors with mixed entries between active directions and ratios $G_{i,mm}/(G_{i,mm} + G_{e,mm})$ that differ from $\kappa$ by more than $10^{-6}\kappa$;
- verifier `manufacturedFDABidomainVerifier`.

## 2.3 Construction

The spatial factor $F_d$ of the FDA monodomain problem is an eigenfunction of the diagonal operator, $\nabla \cdot (\mathbf{G}_i \nabla F_d) = -k_d F_d$ with

$$
k_d = \sum_{m=1}^{d} G_{i,mm} \, (m\pi)^2
$$

($\mathbf{G}_i$ is the conductivity of the monodomain FDA problem, so $k_d$ is the eigenvalue of that document). With $\phi_e = -\kappa V_m + c$ and $\kappa(\mathbf{G}_i + \mathbf{G}_e) = \mathbf{G}_i$,

$$\nabla \cdot (\mathbf{G}_i \nabla V_m) + \nabla \cdot ((\mathbf{G}_i + \mathbf{G}_e)\nabla \phi_e) = \nabla \cdot ((\mathbf{G}_i - \kappa(\mathbf{G}_i + \mathbf{G}_e))\nabla V_m) = 0,$$

so the elliptic equation holds with $s_e = 0$. In the $V_m$ equation $\nabla \cdot (\mathbf{G}_i \nabla \phi_i) = (1-\kappa)\nabla \cdot (\mathbf{G}_i \nabla V_m) = -(1-\kappa)k_d V_m$, and $(1-\kappa)G_{i,mm} = H_{mm}$, the harmonic combination of the two diagonals. The ionic model sets

$$\beta_d = -\sum_{m=1}^{d} H_{mm}\,(m\pi)^2, \qquad H_{mm} = \frac{G_{i,mm}\,G_{e,mm}}{G_{i,mm} + G_{e,mm}}$$

so that $-\beta_d V_m$ from the ionic term (Section 1.2) cancels the diffusion term and no applied current is needed ($I_{\text{app}} = 0$). The constant $c$ makes $\phi_e(\mathbf{x}_{\text{ref}}, t) = \phi_{e,\text{ref}}$ at every time:

$$c = \kappa V_m(\mathbf{x}_{\text{ref}}, t) + \phi_{e,\text{ref}}$$

## 2.4 Exact solution

$$F_1 = \cos(\pi x)$$

$$F_2 = \cos(\pi x)\cos(2\pi y)$$

$$F_3 = \cos(\pi x)\cos(2\pi y)\cos(3\pi z)$$

$$V_m = F_d\sqrt{t+1}$$

$$\kappa = \frac{G_{i,xx}}{G_{e,xx} + G_{i,xx}}$$

$$\phi_e = -V_m\kappa + c$$

$$\phi_i = V_m(1-\kappa) + c$$

with the auxiliary states $u_1$, $u_2$, $u_3$ and the factors $G_d$ of the monodomain FDA problem:

$$G_3 = xy^2z^3 + 1$$

$$u_1 = G_d(t+1) + V_m$$

$$u_2 = \frac{1}{\sqrt{G_d}(t+1)}$$

$$u_3 = 0$$

($G_1 = x+1$, $G_2 = xy^2+1$.) The full expressions are `fields(d)` of `fda_bidomain_mms.py` and `Vm_d<d>`, `phiE_d<d>`, `phiI_d<d>`, `shift_d<d>` (the constant $c$), `u1_d<d>`, `u2_d<d>`, `u3_d<d>`, `Iion_d<d>` (the current $J_{\text{ion}}$), `F_d<d>`, `G_d<d>` of `reference/fda_bidomain.py`, whose `PARAMS` holds $\kappa$, $\beta_d$, the diagonals and the gauge.

## 2.5 Self-checks

`fda_bidomain_mms.py --self-check` asserts, for $d = 1, 2, 3$, in exact rational arithmetic on the decimals of the conductivities:

- $\partial_t u_1$ and $\partial_t u_2$ equal the ionic rates on the exact fields;
- the residuals of both equations are constant multiples $\rho_p V_m$, $\rho_e V_m$ of $V_m$ with $|\rho_p|/k_d \le 10^{-8}$ and $|\rho_e|/k_d \le 10^{-8}$ (the values are printed above);
- $\mathbf{n} \cdot \mathbf{G}_i \nabla V_m$, $\mathbf{n} \cdot \mathbf{G}_e \nabla \phi_e$ and $\mathbf{n} \cdot \mathbf{G}_i \nabla \phi_i$ vanish on every face;
- $\phi_e(\mathbf{x}_{\text{ref}}, t) = \phi_{e,\text{ref}}$;
- the ratios $G_{i,mm}/(G_{i,mm} + G_{e,mm})$ agree with $\kappa$ to $10^{-6}\kappa$, the tolerance of the ionic model;
- the printed constants round-trip.

`docs/bidomain/bidomain_reference.py --self-check` asserts that the conductivities and `phiERefPoint` of the tutorial `constant/electroProperties` equal those of the generator.

*Comparison with a run.* `bidomain_reference.py --run-check fda=CASE` reads the cell centres and volumes (from `constant/polyMesh`, by the face-triangle and face-pyramid decomposition OpenFOAM uses), the computed `Vm` and `phiE` of the last time and `postProcessing/<d>D_<n>_cells.dat`, and recomputes from the exact fields above the `Vm`, `phiE`, `phiE_gauge` and `phiI_gauge` norms and $\delta_e$. On the runs `1D_N10`, `2D_N10` and `3D_N10` of the study `cartesianConvergence` ($t = 0.2$) all agree with the reported values to the six printed digits, and the written initial `preProcess_u1` equals $u_1(\mathbf{x}_c, 0)$ to $2 \times 10^{-15}$.

## 2.6 Figures

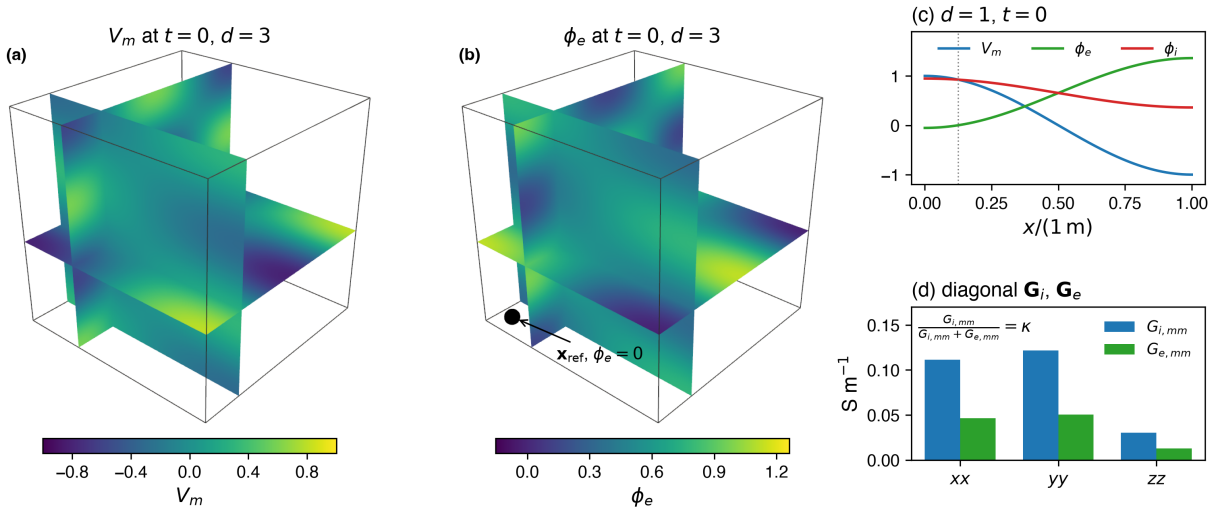
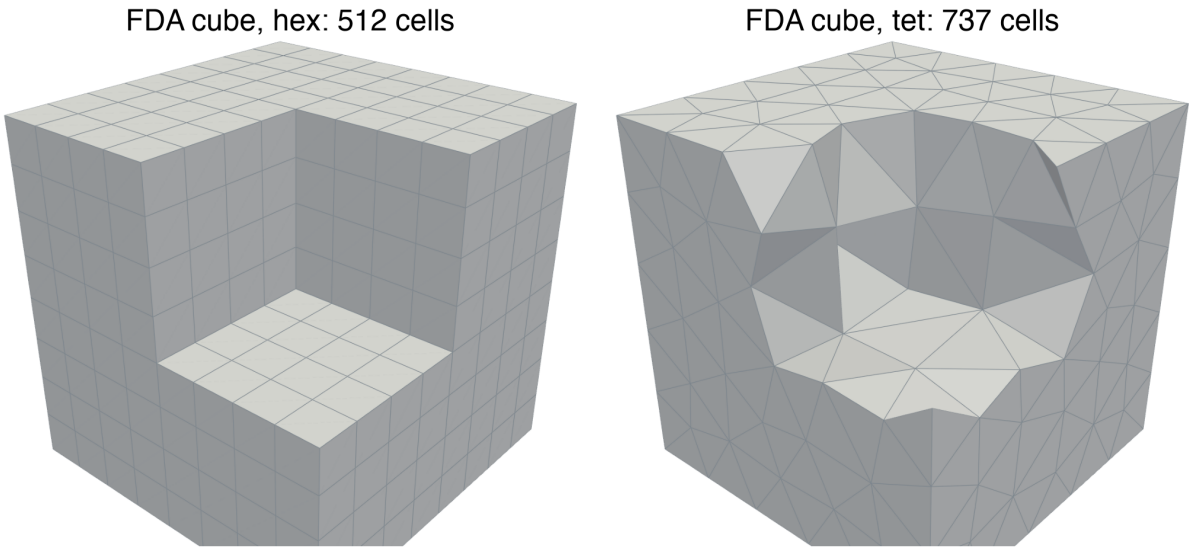



**Figure 1:** FDA cube. (a) $V_m = F_3$ at $t = 0$ on three orthogonal slices. (b) $\phi_e$ at $t = 0$ on the same slices, with $c$ fixed by $\phi_e(\mathbf{x}_{\text{ref}}) = 0$; the dot marks $\mathbf{x}_{\text{ref}}$. (c) $V_m$, $\phi_e = -\kappa V_m + c$ and $\phi_i = (1 - \kappa)V_m + c$ for $d = 1$ at $t = 0$; the dotted line is $x_{\text{ref}}$, where $\phi_e = 0$ and $\phi_i = V_m$. (d) The diagonals of $\mathbf{G}_i$ and $\mathbf{G}_e$, in the same ratio on each axis.




**(a)** hex: `-mesh hex -N 8` **(b)** tet: `-mesh tet -N 5`

**Figure 2:** Meshes of the unit cube from the tutorial `manufacturedSolutions/bidomain` (`-dim 3`), made by the meshing steps of its `Allrun` at a coarse resolution chosen for legibility (the tutorial default is $N = 20$): (a) `blockMesh` with `system/blockMeshDict.3D` and $N = 8$ cells per unit length; (b) `gmsh` Delaunay tetrahedra from `box.geo.template` of element size 1/5, seed 1, converted by `gmshToFoam`. The octant facing the viewer is removed cell by cell.

### 2.7 Regeneration

```
python fda_bidomain_mms.py --emit
python fda_bidomain_mms.py --self-check
python docs/bidomain/bidomain_reference.py --self-check
R=../bidomain/setup/studies/cartesianConvergence/results
python docs/bidomain/bidomain_reference.py --run-check fda=$R/3D_N10
```

`--emit` writes `reference/fda_bidomain.py` and `latex/fda_bidomain_*.tex`. The spatial factors, the auxiliary states and the ionic closure are those of `fda_monodomain_mms.py`, imported. The bidomain functions of cardiacFoam (`manufacturedFDABidomainReference.H` and the constants of `bidomainFDAManufactured.C`) are hand-written and carry no generated regions, so the generator has no `--check`; the run comparison above is the comparison.

## 3 Sealed spherical shell

### 3.1 Description and limitations

The heart-like shell $r_1 < |\mathbf{x}| < r_2$ of the monodomain document, both spheres sealed, with two full intracellular and extracellular tensors whose fibre and sheet rotate through the wall. The tensors share one orthonormal frame but have unequal eigenvalue ratios, so $\mathbf{G}_i + \mathbf{G}_e$ is not a multiple of $\mathbf{G}_i$, and both are oblique to both walls. The exact $\phi_i$ and $\phi_e$ have non-zero normal and tangential gradients on both walls while $\mathbf{n} \cdot \mathbf{G}_i \nabla \phi_i$ and $\mathbf{n} \cdot \mathbf{G}_e \nabla \phi_e$ vanish there exactly. The case exercises the sealed-wall operators of both potentials on curved walls, a non-trivial $\phi_e$ source with zero integral, the pure Neumann $\phi_e$ system with its gauge and compatibility, and spatially varying tensors whose derivatives enter both sources.

It does not exercise a bath, the ECG, a stimulus or a wall that carries current, and the two tensors have the same principal axes everywhere, so a discretisation error that appears only when the eigenvectors of $\mathbf{G}_i$ and $\mathbf{G}_e$ differ is not detected.

### 3.2 Problem instance

The parameters are

$$
\begin{gathered}
r_1 = 0.5, \quad r_2 = 1, \quad \gamma_1 = 20^\circ, \quad \gamma_2 = 50^\circ, \quad k = \pi, \\
\sigma_{i,f} = 0.17, \quad \sigma_{i,s} = 0.019, \quad \sigma_{i,n} = 0.01, \quad \sigma_{e,f} = 0.62, \quad \sigma_{e,s} = 0.12, \\
\sigma_{e,n} = 0.08, \quad \theta_0 = -20^\circ, \quad \theta_1 = 90^\circ, \quad a_1 = -0.1, \quad a_2 = 0.3, \\
a_3 = 0.25, \quad b_1 = 0.3, \quad b_2 = 0.2, \quad b_3 = 0.4, \quad b_4 = 0.15
\end{gathered}
$$

with $\chi = 3$, $C_m = 2$, and the gauge and reaction coefficient

$$
\mathbf{x}_{\mathrm{ref}} = (0.6, 0.3, 0.2)\,, \quad \phi_{e,\mathrm{ref}} = 0, \quad \beta = -8.6
$$

The inner sphere is the patch `endocardium` and the outer sphere the patch `epicardium`. In cardiacFoam:

- tutorial `manufacturedSolutions/bidomain/sphericalShell`, options `-mesh hex|tet` and `-N`;
- myocardium solver `bidomainSolver` (`sealedHeartBoundary true`, `conductivitySource field`, the cell tensors written by `setManufacturedFields` to `0/ConductivityIntracellular` and `0/ConductivityExtracellular`);
- ionic model `monodomainFDAManufactured` (`dimension "3D"`);
- verifier `manufacturedShellBidomainVerifier`, which adds $I_{\text{app}}$ and $s_e$ and requests the compatibility projection of the $\phi_e$ system.

## 3.3 Construction

*Transmural coordinate and conductivities.* The coordinate $\lambda$, the fibre angle $\theta$, the fixed frame $(\mathbf{e}_a, \mathbf{e}_b, \mathbf{n}_s)$, the fibre $\mathbf{f}$ and the sheet $\mathbf{s}$ are those of the monodomain shell:

$$\lambda = \frac{-r_1^2 + |\mathbf{x}|^2}{-r_1^2 + r_2^2}$$

$$\theta = \lambda\theta_1 + \theta_0$$

$$[\mathbf{e}_a\ \mathbf{e}_b\ \mathbf{n}_s] = \begin{bmatrix} \cos(\gamma_1) & -\sin(\gamma_1) & 0 \\ \sin(\gamma_1) & \cos(\gamma_1) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\gamma_2) & -\sin(\gamma_2) \\ 0 & \sin(\gamma_2) & \cos(\gamma_2) \end{bmatrix}$$

$$\mathbf{f} = \mathbf{e}_a \cos(\theta) + \mathbf{e}_b \sin(\theta)$$

$$\mathbf{s} = -\mathbf{e}_a \sin(\theta) + \mathbf{e}_b \cos(\theta)$$

Both tensors are built on this frame with their own eigenvalues:

$$\mathbf{G}_i = \mathbf{f} \otimes \mathbf{f}\sigma_{i,f} + \sigma_{i,n}\mathbf{n}_s \otimes \mathbf{n}_s + \sigma_{i,s}\mathbf{s} \otimes \mathbf{s}$$

$$\mathbf{G}_e = \mathbf{f} \otimes \mathbf{f}\sigma_{e,f} + \sigma_{e,n}\mathbf{n}_s \otimes \mathbf{n}_s + \sigma_{e,s}\mathbf{s} \otimes \mathbf{s}$$

$\mathbf{G}_i$ is the conductivity $\boldsymbol{\sigma}$ of the monodomain shell. The eigenvalue ratios are $17 : 1.9 : 1$ for $\mathbf{G}_i$ and $7.75 : 1.5 : 1$ for $\mathbf{G}_e$.

*Sealing.* Each potential is sealed with respect to its own tensor by the construction of the monodomain shell. $\omega$ vanishes on both spheres, with normal derivative $d_j$ on sphere $j$ and $\mathbf{n}_j$ the unit normal out of the heart:

$$\omega = \frac{\left(-r_1^2 + |\mathbf{x}|^2\right)\left(-r_2^2 + |\mathbf{x}|^2\right)}{(r_1 + r_2)\left(-r_1^2 + r_2^2\right)}$$

$$\mathbf{n}_1 = -\frac{\mathbf{x}}{r_1}$$

$$\mathbf{n}_2 = \frac{\mathbf{x}}{r_2}$$

$$d_1 = \frac{2r_1}{r_1 + r_2}$$

$$d_2 = \frac{2r_2}{r_1 + r_2}$$

For the intracellular factor, with $\mathbf{G}_i$,

$$m_{i,j} = -\frac{\left(\mathbf{n}_j \cdot \mathbf{G}_i \nabla \hat{h}_i\right)}{\left(\mathbf{n}_j \cdot \mathbf{G}_i \, \mathbf{n}_j\right) d_j}$$

$$m_i = \lambda m_{i,2} + m_{i,1} \, (1 - \lambda)$$

$$F_i = \hat{h}_i + m_i \omega$$

and for the extracellular factor, with $\mathbf{G}_e$,

$$m_{e,j} = -\frac{\left(\mathbf{n}_j \cdot \mathbf{G}_e \nabla \hat{h}_e\right)}{\left(\mathbf{n}_j \cdot \mathbf{G}_e \, \mathbf{n}_j\right) d_j}$$

$$m_e = \lambda m_{e,2} + m_{e,1} \, (1 - \lambda)$$

$$F_e = \hat{h}_e + m_e \omega$$

so that $\mathbf{n}_j \cdot \mathbf{G}_i \nabla F_i = 0$ and $\mathbf{n}_j \cdot \mathbf{G}_e \nabla F_e = 0$ on both spheres. $F_i$ equals the field $F$ of the monodomain shell.

*Potentials and gauge.* $\phi_i = \sqrt{1+t}\, F_i + c$ and $\phi_e = \sqrt{1+t}\, F_e + c$, so $V_m = \sqrt{1+t}\, (F_i - F_e)$ carries no constant, and $c$ makes $\phi_e(\mathbf{x}_{\text{ref}}, t) = \phi_{e,\text{ref}}$:

$$c = \phi_{e,\text{ref}} - F_e(\mathbf{x}_{\text{ref}}) \sqrt{t+1}$$

*Ionic closure and sources.* The auxiliary states use the positive function $g$ of the monodomain shell, so the ionic rates hold exactly and the ionic term leaves $-\beta V_m$ in the $V_m$ equation (Section 1.2). The applied current is the remainder of the $V_m$ equation and $s_e$ is the residual of the $\phi_e$ equation with no source:

$$I_{\text{app}} = \sqrt{t+1} \, (\beta \, (-F_e + F_i) - \nabla \cdot (\mathbf{G}_i \nabla F_i))$$

$$s_e = (\nabla \cdot (\mathbf{G}_e \nabla F_e) + \nabla \cdot (\mathbf{G}_i \nabla F_i)) \sqrt{t+1}$$

Both fluxes $\mathbf{G}_i \nabla F_i$ and $\mathbf{G}_e \nabla F_e$ have zero normal component on both spheres, so by the divergence theorem $\int_\Omega s_e \, \mathrm{d}V = 0$: the $\phi_e$ problem is compatible in the continuum. The divergences include the derivatives of the spatially varying tensors.

### 3.4 Exact solution

$$\hat{h}_i = a_1 z + a_2 \left(1 - \frac{3x^2}{10}\right) \cos(ky) + a_3 x \sin(kz)$$

$$\hat{h}_e = b_1 x + b_2 yz + b_3 \left(\frac{y}{2} + 1\right) \sin(kx) + b_4 e^{\frac{2z}{5}} \cos\left(ky + \frac{3}{10}\right)$$

$$\phi_i = F_i \sqrt{t+1} + c$$

$$\phi_e = F_e \sqrt{t+1} + c$$

$$V_m = (-F_e + F_i) \sqrt{t+1}$$

$$g = \frac{\sin\left(kx + \frac{1}{5}\right) \cos(kz) \cos\left(ky - \frac{1}{3}\right)}{4} + 1$$

$$u_1 = V_m + g\,(t+1)$$

$$u_2 = \frac{1}{\sqrt{g}\,(t+1)}$$

with $u_3 = 0$. The divergences are not printed: they are `expressions()["divGiGradFi"]` and `expressions()["divGeGradFe"]` of `shell_bidomain_mms.py` and `divGiGradFi`, `divGeGradFe` of `reference/shell_bidomain.py`, which also holds `Fi`, `Fe`, `gradFi`, `gradFe`, `Gi`, `Ge` and `g`. The cardiacFoam header `manufacturedShellBidomainReference.H` is written whole by the generator.

### 3.5 Self-checks

`shell_bidomain_mms.py --self-check` asserts:

- sealing: $\max|\mathbf{n} \cdot \mathbf{G}_i \nabla F_i|$ and $\max|\mathbf{n} \cdot \mathbf{G}_e \nabla F_e|$ below $10^{-25}$ at 200 points of each sphere, in 30-digit arithmetic on the exact sphere;
- sources: both divergences agree with central differences of the fluxes to $10^{-6}$ at 40 points of the shell;
- positivity: every $\mathbf{n}_j \cdot \mathbf{G}\,\mathbf{n}_j > 0$ and $g > 0.5$ over 4000 points of the shell widened by 0.05 on each side;
- richness of the walls: on each wall the means of the tangential parts of $\mathbf{G}_i\mathbf{n}/\sigma_{i,f}$, $\mathbf{G}_e\mathbf{n}/\sigma_{e,f}$, $\nabla F_i$, $\nabla F_e$ and $\nabla(F_i - F_e)$ exceed $10^{-2}$;
- compatibility: the Gauss–Legendre integral of $s_e/\sqrt{1+t}$ over the shell ($60 \times 60$ points in $|\mathbf{x}|$ and $\cos\vartheta$, 120 in the azimuth) is below $10^{-12}$ times the integral of its absolute value;
- $F_i$ equals the monodomain shell field to $10^{-13}$ at 200 points;
- $\mathbf{G}_i$ and $\mathbf{G}_e$ commute to $10^{-14}$ (one frame) at 500 points, and the relative distance of $\mathbf{G}_e$ from the multiples of $\mathbf{G}_i$ is at least 0.1 at every point (it is 0.104 at least);
- $\mathbf{G}_i$ equals $\boldsymbol{\sigma}$ of `shell_monodomain_mms.py`;

- the residual of the $V_m$ equation and of the two ionic rates vanish symbolically, with $F_i$, $F_e$, $g$, $\beta$ and $\nabla \cdot (\mathbf{G}_i \nabla F_i)$ as symbols;
- the reference point $\mathbf{x}_{\mathrm{ref}}$ lies inside the shell.

`docs/bidomain/bidomain_reference.py --self-check` asserts that `phiERefPoint` and the ionic model of the tutorial equal those of the generator.

*Comparison with a run.* `bidomain_reference.py --run-check shell=CASE` recomputes, from the cell centres and volumes of `constant/polyMesh`, the tensors and the `Vm` and `phiE_gauge` norms of a finished run and compares them with `0/ConductivityIntracellular`, `0/ConductivityExtracellular` and `postProcessing/3D_<hex|tet>_<N>_cells.dat`. On the runs `sealed_hex_N8_s1` (study `hexConvergence`) and `sealed_tet_N10_s1` (study `tetConvergence`), both at $t = 0.2$, the tensors agree to $3 \times 10^{-15}$ relative, the norms to $5 \times 10^{-12}$ relative (12 printed digits), and `preProcess_u1` equals $u_1(\mathbf{x}_c, 0)$ to $10^{-15}$.

## 3.6 Figures

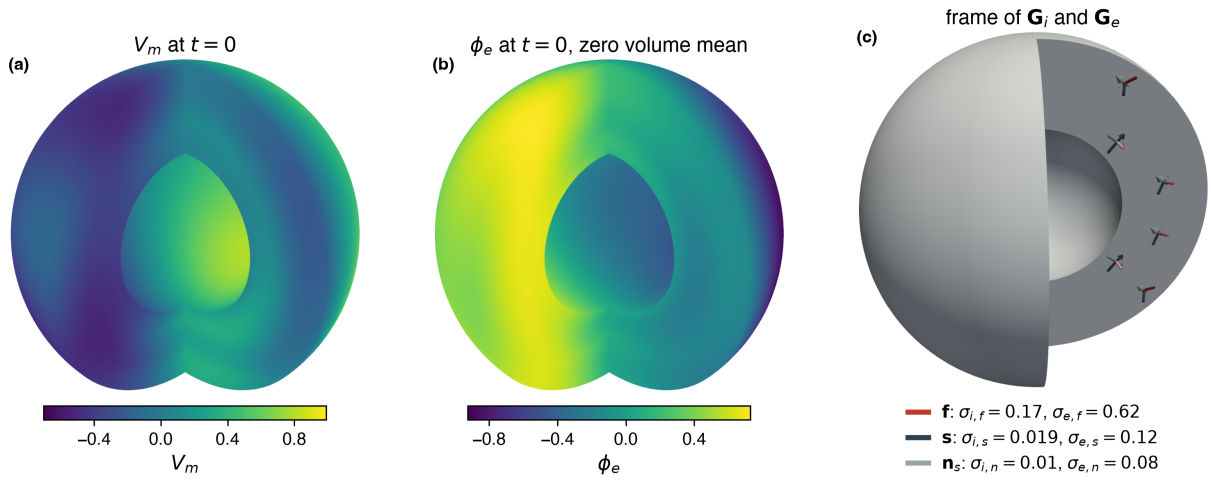



**Figure 3:** Sealed shell. (a) $V_m = F_i - F_e$ at $t = 0$ on a cut-away of the shell. (b) $\phi_e$ at $t = 0$ with its volume mean over the shell removed, the gauge of the `phiE_gauge` rows. (c) The frame common to $\mathbf{G}_i$ and $\mathbf{G}_e$ at six sites through the wall on the cut face, the fibre shaded from the inner (light) to the outer (dark) wall, with the eigenvalues of each tensor along each direction.

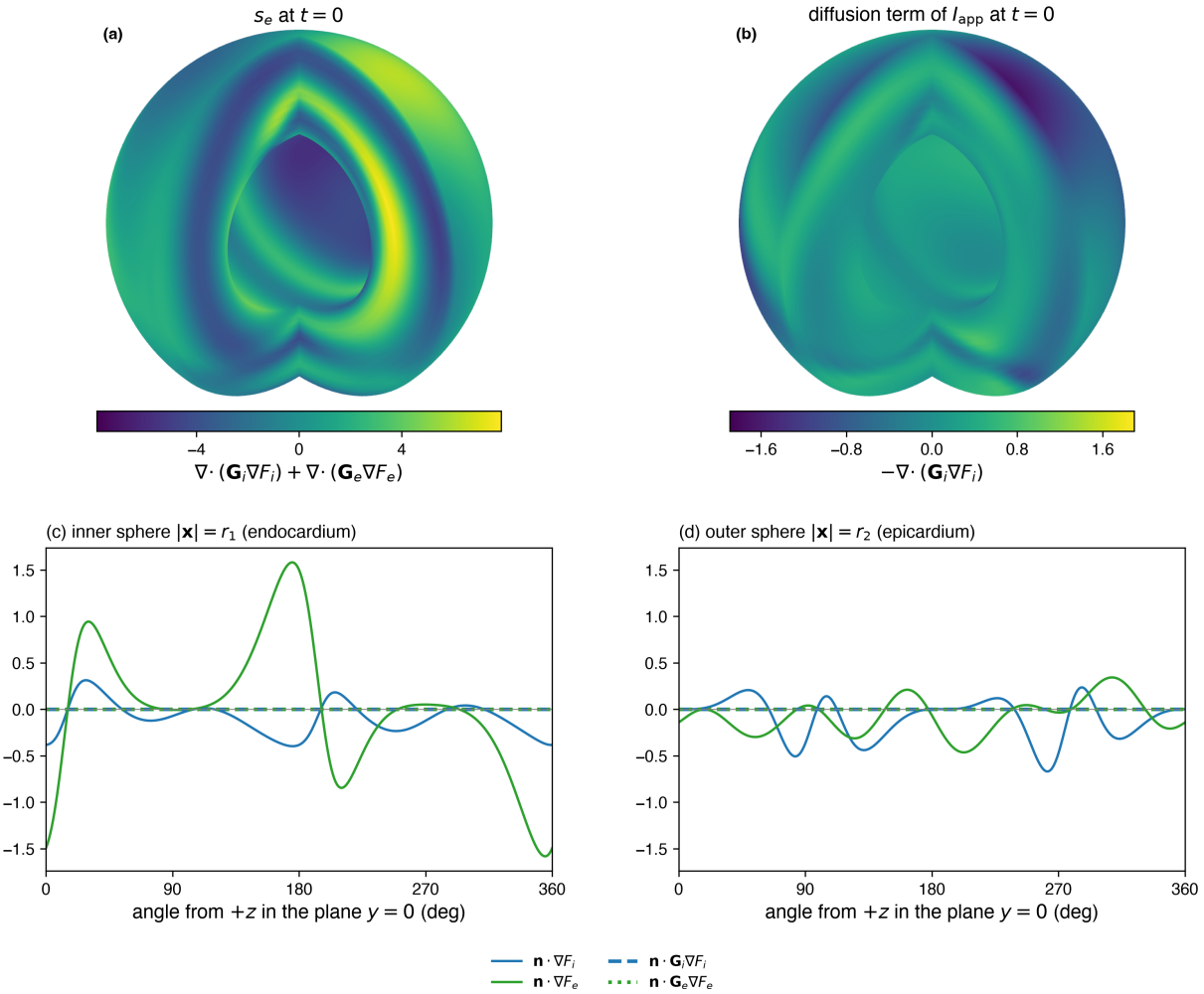
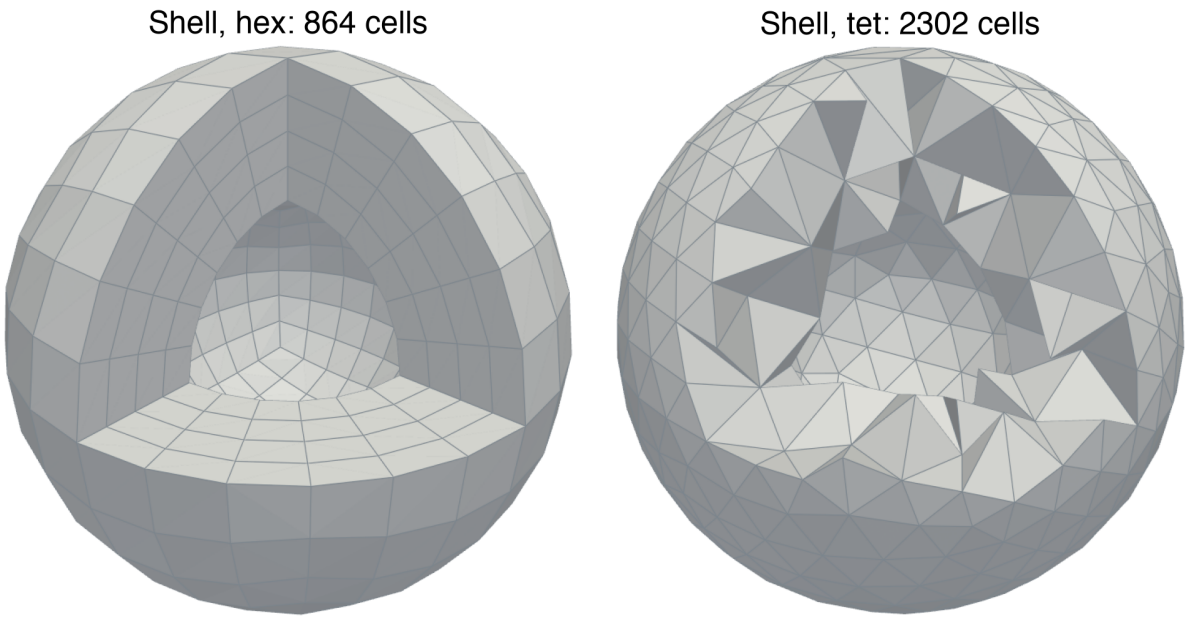



**Figure 4:** Sources and walls of the shell. (a) $s_e/\sqrt{1+t}$, the source of the $\phi_e$ equation. (b) The diffusion term of $I_{\text{app}}/\sqrt{1+t}$, $-\nabla\cdot(\mathbf{G}_i\nabla F_i)$. (c), (d) On the great circle of each sphere in the plane $y = 0$: the normal derivatives $\mathbf{n}\cdot\nabla F_i$ and $\mathbf{n}\cdot\nabla F_e$ (solid) are non-zero, the conormal fluxes $\mathbf{n}\cdot\mathbf{G}_i\nabla F_i$ and $\mathbf{n}\cdot\mathbf{G}_e\nabla F_e$ (broken) vanish to rounding ($\mathbf{n}$ out of the heart).




**(a)** hex: `-mesh hex -N 8`

**(b)** tet: `-mesh tet -N 5`

**Figure 5:** Meshes of the shell from the tutorial `manufacturedSolutions/bidomain/sphericalShell`, made by the meshing steps of its `Allrun` at a coarse resolution chosen for legibility (the tutorial default is $N = 12$): (a) the `blockMesh` cubed sphere with $N = 8$; (b) `gmsh` Delaunay tetrahedra from `shell.geo.template` of element size $1/5$, seed 1, converted by `gmshToFoam`. The octant facing the viewer is removed cell by cell, exposing the cavity.

## 3.7 Regeneration

```
python shell_bidomain_mms.py --emit
python shell_bidomain_mms.py --self-check
H=../../../src/verificationModels/bidomainVerification
python shell_bidomain_mms.py --check $H/manufacturedShellBidomainReference.H
R=../bidomain/sphericalShell/setup/studies/hexConvergence/results
python docs/bidomain/bidomain_reference.py --run-check shell=$R/sealed_hex_N8_s1
```

`--emit` writes `reference/shell_bidomain.py` and `latex/shell_bidomain_*.tex`; the shared fragments `latex/shell_*.tex` are those of `shell_monodomain_mms.py`. `--write` writes the whole cardiacFoam header, including a table of self-test samples that the verifier compares with its compiled functions at start-up.

## References

Pathmanathan, Pras, and Richard A. Gray. 2014. "Verification of Computational Models of Cardiac Electro-Physiology." *International Journal for Numerical Methods in Biomedical Engineering* 30 (5): 525–44. https://doi.org/10.1002/cnm.2615.

Roache, Patrick J. 2002. "Code Verification by the Method of Manufactured Solutions." *Journal of Fluids Engineering* 124 (1): 4–10. https://doi.org/10.1115/1.1436090.

U.S. Food and Drug Administration, Center for Devices and Radiological Health. 2023. "Verification Test Problems for Cardiac Electrophysiology Modeling Software." Catalog of Regulatory Science Tools. https://cdrh-rst.fda.gov/verification-test-problems-cardiac-electrophysiology-modeling-software.